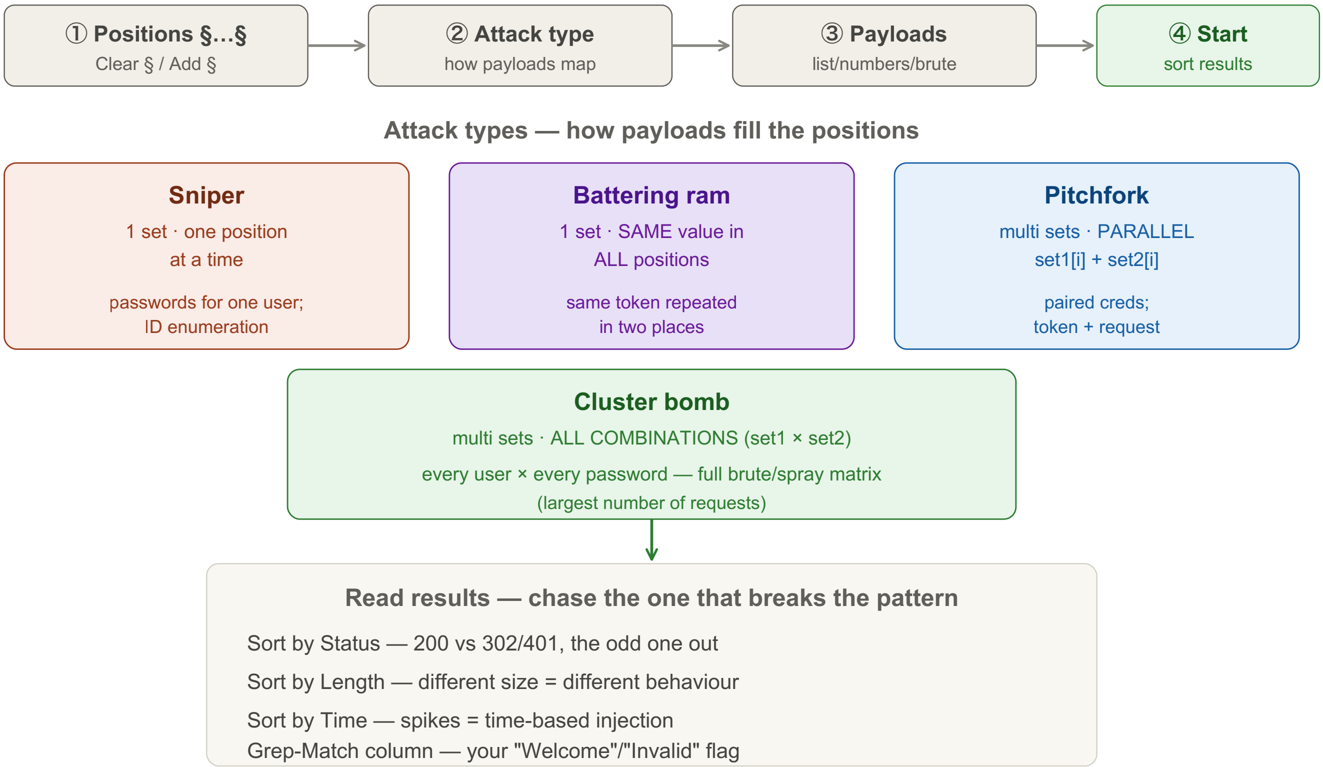
Burp Intruder

Automates sending a request many times with varying values injected at positions you choose — fuzz parameters, brute-force, enumerate IDs. Repeater is ONE request by hand; Intruder is MANY automatically. Replace <placeholders> with your own values.



① Send to Intruder & Set Positions

Mark where payloads get injected with the § markers.

Tab 1 · Intruder

```
right-click request → "Send to Intruder"  (Ctrl+I)

"Clear §"               remove all auto-marked positions
select a value → "Add §"   mark what to fuzz

# example — fuzz one parameter
GET /api/user?id=§1§ HTTP/1.1
```

---

② Pick the Attack Type

This decides how payloads map onto positions — the most important choice.

Tab 2 · Attack type

```
Sniper        1 set, ONE position at a time   → single-param fuzz, ID enum
Battering ram 1 set, SAME value ALL positions → reuse one value in many spots
Pitchfork     multi sets, PARALLEL set1[i]+set2[i] → paired creds, token+req
Cluster bomb  multi sets, ALL COMBINATIONS set1×set2 → full user×pass matrix
```

Cluster bomb explodes fast: 100 users × 100 passwords = 10,000 requests.

---

③ Load Payloads

Each position gets a payload set + type, with optional processing.

Tab 3 · Payloads

```
Payload set (which position) → Payload type:
  Simple list   paste / import a wordlist
  Numbers       ranges (1000-2000) for ID enumeration
  Runtime file  stream a large wordlist from disk
  Brute forcer  char sets + length

Payload processing: prefix/suffix, hash, Base64-encode, etc.
Keep "URL-encode these characters" ON for safety.
```

---

④ Grep-Match, Run & Read

Flag interesting responses, then sort to find the outlier.

Tab 4 · Results

```
Settings → Grep - Match: flag responses containing a string
  e.g. "Welcome", "Invalid", "admin", an error signature

"Start attack" → sort the results table by:
  Status  (200 vs 302/401)   Length (size change)
  Time    (timing spikes)    Grep column (your flag)
→ the result that BREAKS the pattern is usually the finding
```

Use cases: login brute/spray → Cluster bomb (users×pass); IDOR → Sniper + Numbers; hidden param → Sniper + wordlist watching Length; token testing → Pitchfork.

Caveats

Tab 1 · Notes

```
# Community Edition throttles Intruder to ~1 req/sec.
# For heavy brute force use ffuf / wfuzz / hydra, then confirm the hit in Burp.
# Set Resource pool throttle/delay to avoid lockouts & rate limits.
```

Only test authorized targets.

Key idea: Intruder = request template + payload positions (§) + an attack type that decides how payloads fill those positions. Sniper for one knob, Cluster bomb for the full matrix, Pitchfork for paired lists. You don't read every response — you sort by status/length/time/grep and chase the one that breaks the pattern. For large brute force, a dedicated fuzzer beats Community-Edition Intruder.